

## Informations Générales

- Nom de l'étudiant : KHAMRICH Mohammed
- Encadre par : Pr : B.ELBAGHAZAOUI

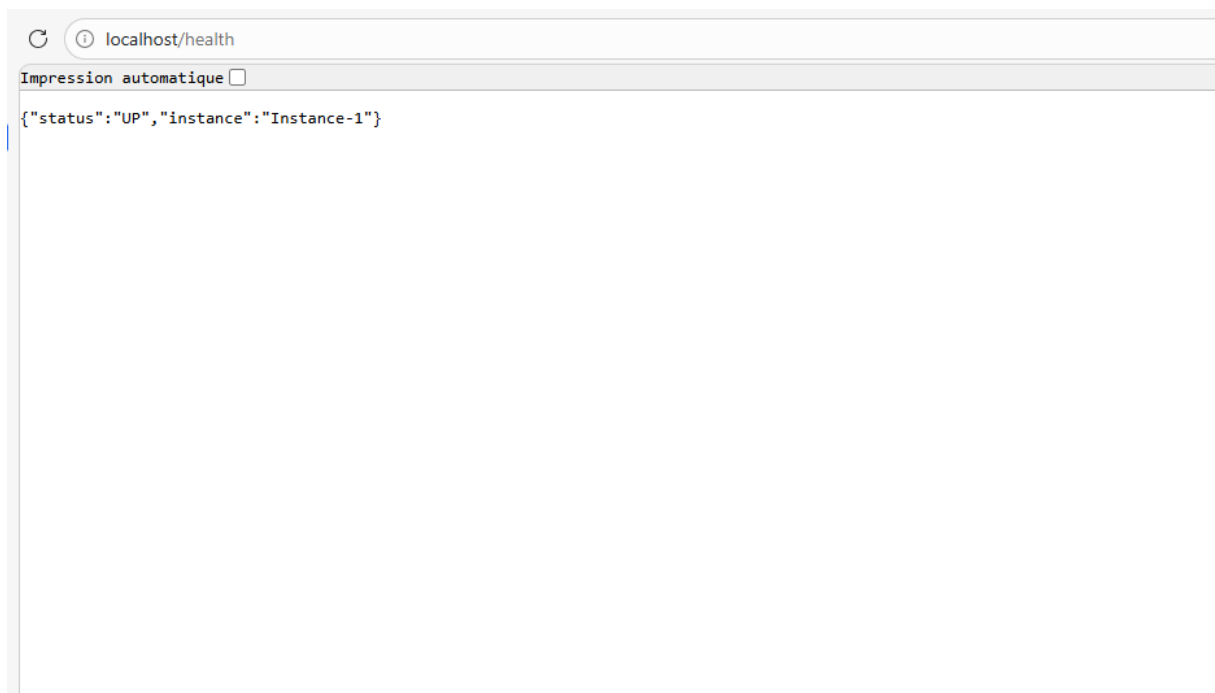
## TP3 : Architecture distribuée

### Objectifs

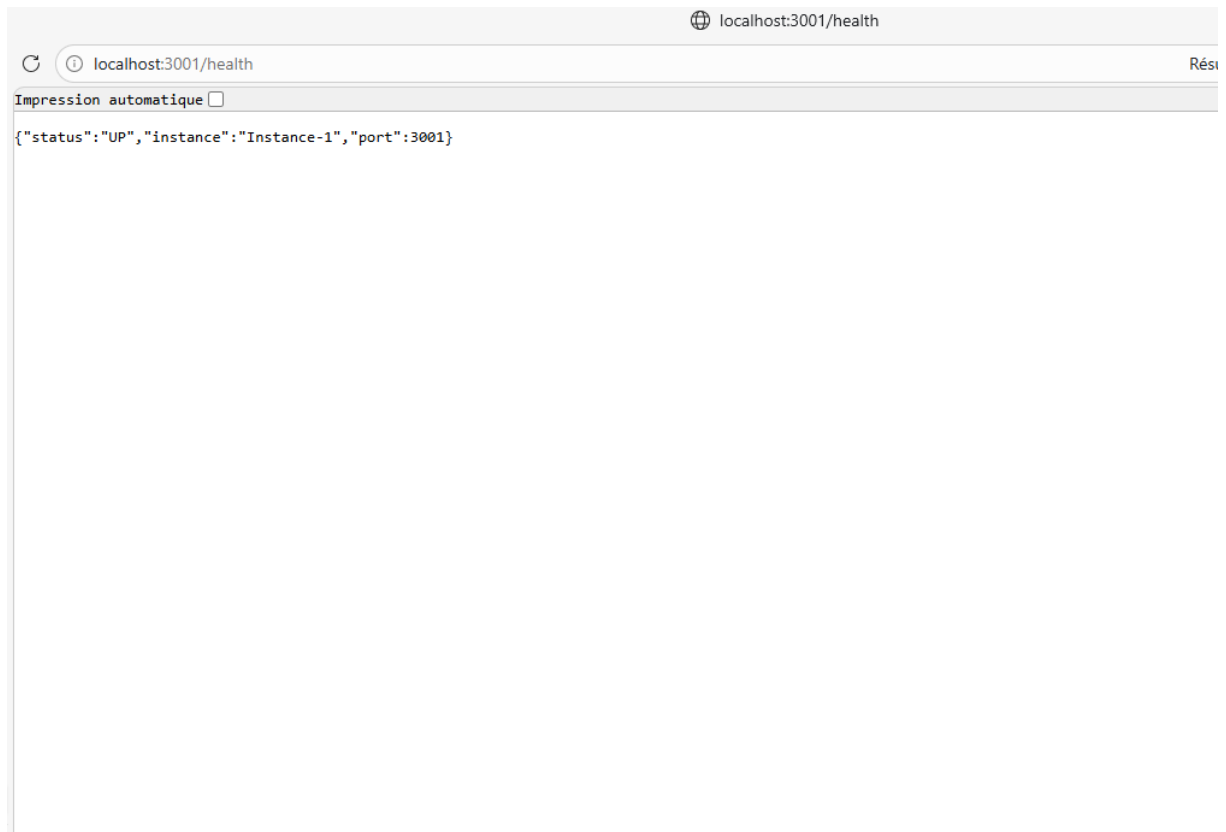
Déployer une architecture distribuée complète avec 6 composants interconnectés. Comprendre le rôle de chaque composant : Client, Load Balancer, Serveur, Cache, BDD, Bus de messages. Implémenter un flux complet : POST commande → stockage MongoDB → cache Redis → notification RabbitMQ. Tester et observer le comportement distribué avec Postman.

1-

[localhost/health](http://localhost/health)



## [localhost:3001/health](http://localhost:3001/health)

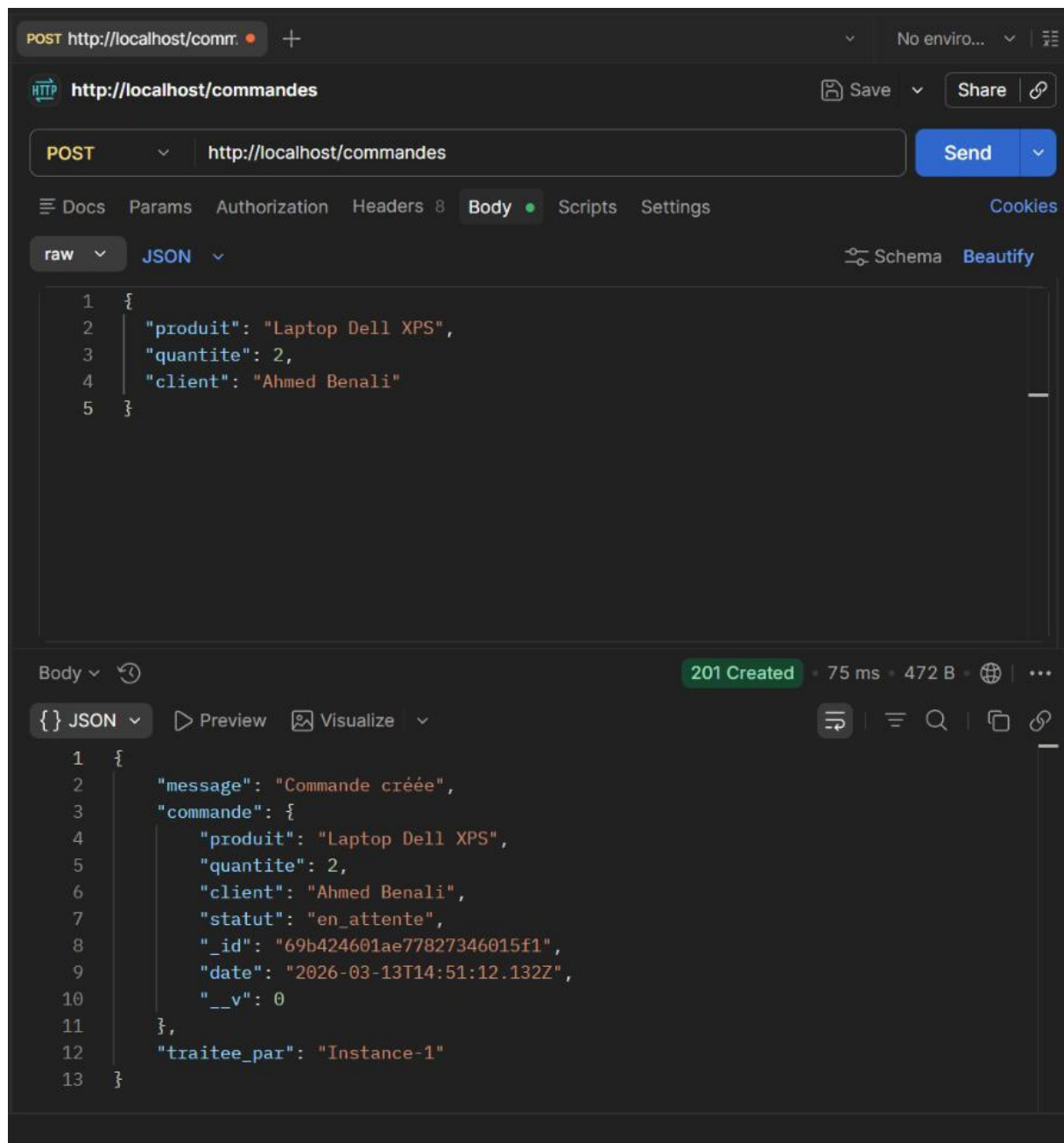


## [localhost:3002/health](http://localhost:3002/health)



## Test avec Postman :

### Test1 :



### Test2 :

GET http://localhost/comma + No enviro...

HTTP http://localhost/commandes Save Share

GET http://localhost/commandes Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

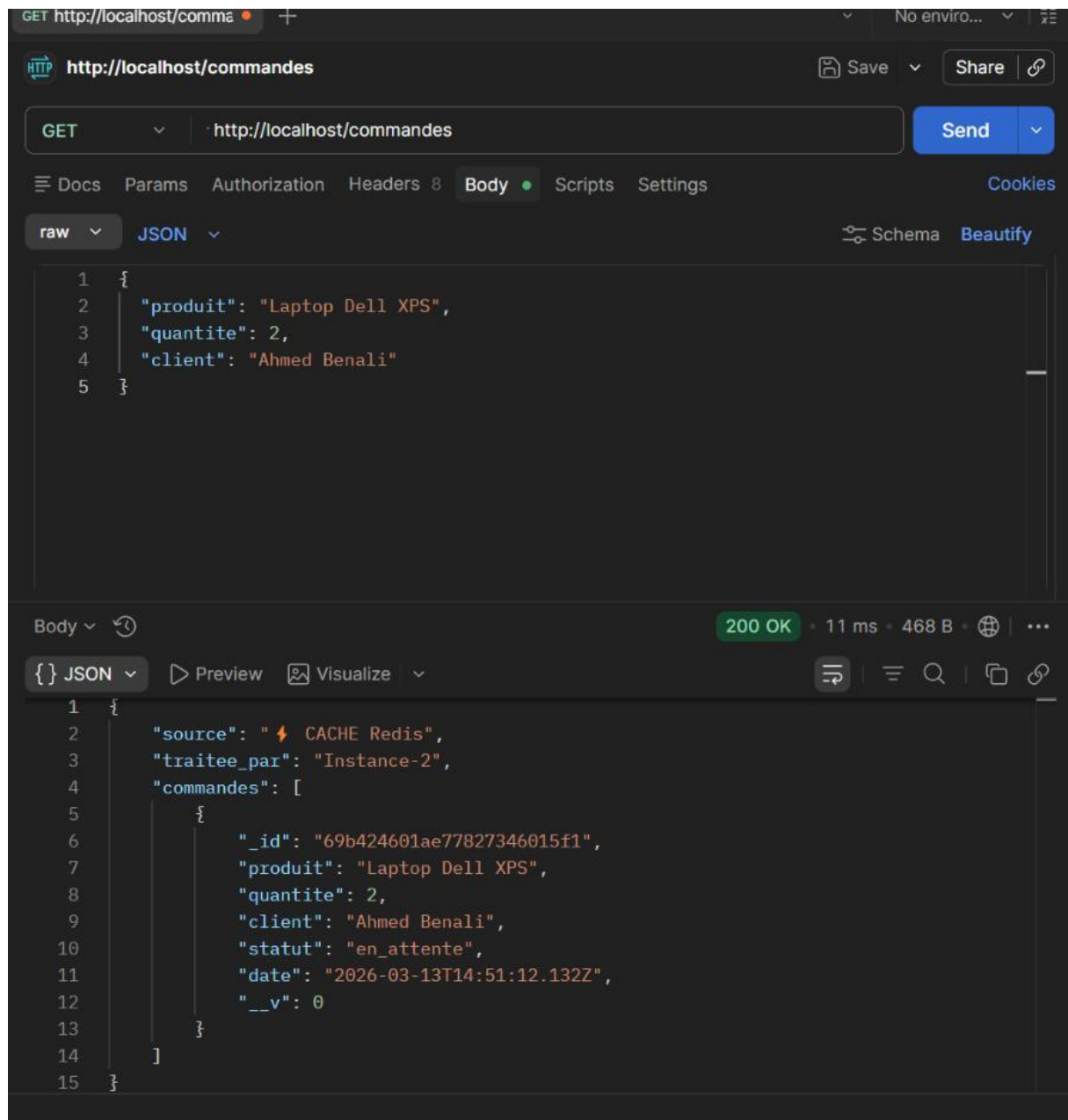
```
1 {
2   "produit": "Laptop Dell XPS",
3   "quantite": 2,
4   "client": "Ahmed Benali"
5 }
```

Body 200 OK 36 ms 465 B

{ } JSON Preview Visualize

```
1 {
2   "source": "MongoDB",
3   "traitee_par": "Instance-2",
4   "commandes": [
5     {
6       "_id": "69b424601ae77827346015f1",
7       "produit": "Laptop Dell XPS",
8       "quantite": 2,
9       "client": "Ahmed Benali",
10      "statut": "en_attente",
11      "date": "2026-03-13T14:51:12.132Z",
12      "__v": 0
13    }
14  ]
15 }
```

**Test3 :**



**Q1 – Quel composant reçoit en premier les requêtes du client ? Quel algorithme utilise-t-il ?** C'est **Nginx** qui intercepte en premier toutes les requêtes des clients (port 80). Par défaut, il applique l'algorithme **Round-robin**, qui répartit les requêtes successivement entre les différentes instances : requête 1 → app1, requête 2 → app2, requête 3 → app3, etc.

**Q2 – Que se passe-t-il si app1 tombe en panne ? Nginx continue-t-il à fonctionner ? Pourquoi ?** Si **app1** cesse de répondre, Nginx le détecte et redirige automatiquement tout le trafic vers les autres instances disponibles (ex. app2). Le service reste donc opérationnel, car Nginx retire l'instance défaillante du pool jusqu'à son redémarrage.

**Q3 – Différence entre le 1er GET (MongoDB) et le 2ème GET (Redis) :** Lors du **premier GET**, le cache Redis est vide : Node.js interroge directement **MongoDB** et stocke le résultat dans Redis pour 30 secondes. Lors du **deuxième GET** (dans ce délai), Node.js récupère les données directement depuis **Redis**, sans solliciter MongoDB. La réponse est quasi instantanée, ce qui améliore fortement les performances.

**Q4 – Pourquoi invalider le cache Redis après un POST /commande ?** Après la création d'une nouvelle commande, le cache Redis devient obsolète puisqu'il ne contient pas la donnée fraîche. Si on ne le supprime pas, le prochain GET renverrait des informations incomplètes. L'instruction `redis.del('toutes_commandes')` force donc la mise à jour en allant chercher les données directement dans MongoDB.

**Q5 – En quoi RabbitMQ améliore-t-il la résilience du système ?** RabbitMQ assure un découplage entre Node.js et le service de notification. Node.js n'attend pas la confirmation de la notification pour répondre au client. La communication asynchrone évite les blocages et rend le système plus robuste face aux pannes partielles.

**Q6 – Si on voulait ajouter une 3ème instance Node.js, quelle ligne faudrait-il modifier dans nginx.conf ?** Il suffirait d'ajouter la ligne suivante dans le bloc **upstream** du fichier `nginx/nginx.conf` :

```
server app3:3003;
```